Customer Churn Prediction

Step-by-Step Pipeline Walkthrough

Figure 1 — End-to-end pipeline overview: five stages from raw data to inference

This document explains every stage of the customer churn prediction ML pipeline — from synthetic data generation through model training and evaluation. Two models are compared: **Random Forest** and **XGBoost**, both trained on a 5,000-customer synthetic telecom dataset with SMOTE-balanced classes.

5,000	14 raw features	23 final features	31.6% churn rate
Customers	Before preprocessing	After preprocessing	Realistic imbalance

Step 1 — Synthetic Data Generation

Since no real dataset was available, 5,000 customer records were generated using numpy's statistical distributions. Each feature is sampled from a distribution that mirrors real-world telecom patterns:

- tenure — Exponential(scale=30), clipped to 1–72 months. Most customers churn early or stay long.
- monthly_charges — Normal(65, 25), clipped to 20–150. Gaussian around a mid-tier plan.
- support_calls — Poisson(lambda=1.5). Call events are naturally Poisson-distributed.
- contract_type — Categorical choice: 55% month-to-month, 25% one-year, 20% two-year.
- internet_service — 44% Fiber optic, 34% DSL, 22% No internet.

The churn score formula

Every customer gets a churn_score computed as a weighted sum of their features, then passed through a sigmoid to produce a probability. If P > 0.5, the customer is labelled churned:

```
churn_score = -0.05*tenure + 0.008*monthly_charges + 0.4*(month-to-month) - 0.3*(two-year) +
0.3*(fiber) + 0.25*(electronic check) - 0.2*(tech_support) + 0.08*support_calls + noise(0,
0.3)
```

The weights are grounded in real telecom research: month-to-month contracts (+0.4) have no switching cost; longer tenure (-0.05/month) builds inertia; fiber users (+0.3) are more price-conscious. The noise term ensures a realistic, non-trivially-separable decision boundary.

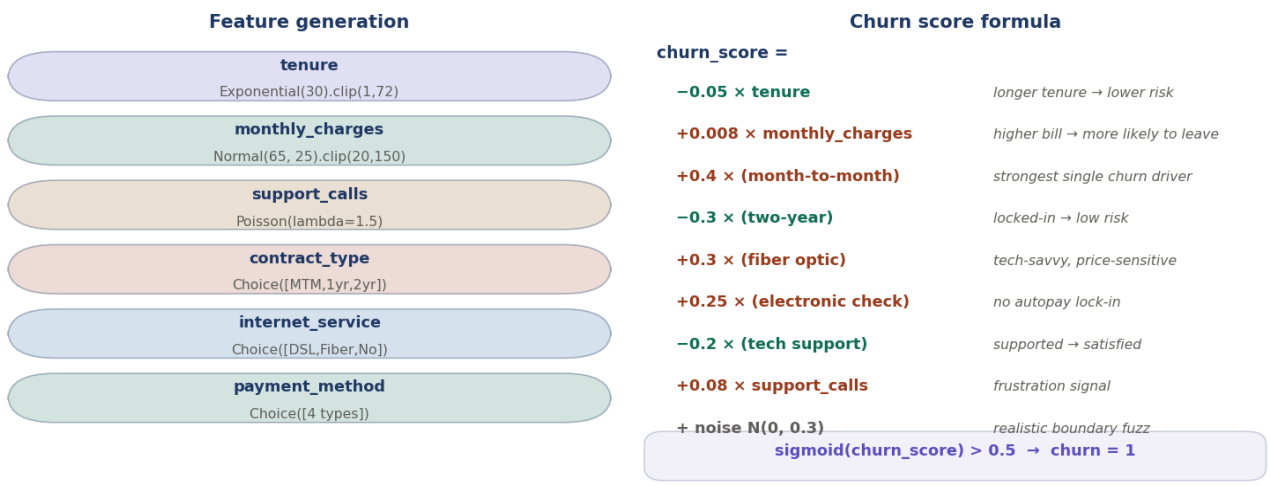

Figure 2 — Feature distributions (left) and churn score formula weights (right)

Step 2 — Preprocessing

ML models operate on numbers. The 14 raw features include 6 categorical text columns that must be converted before training.

Binary encoding

Five Yes/No columns (tech_support, online_security, dependents, partner, paperless_billing) are converted to 1/0 using a simple boolean cast:

```
df[col] = (df[col] == 'Yes').astype(int)
```

One-hot encoding

Three multi-class columns are exploded into separate binary columns using pd.get_dummies. We use drop_first=False — keeping all categories — because tree models don't suffer from multicollinearity, so dropping a dummy is unnecessary:

```
df = pd.get_dummies(df, columns=['contract_type','internet_service','payment_method'],
drop_first=False)
```

This adds 10 new columns: 3 for contract type, 3 for internet service, 4 for payment method.

Feature engineering

- **avg_monthly_spend** = total_charges / (tenure + 1). Captures actual payment history rather than the listed rate — new customers on promotions have lower avg spend.
- **tenure_bucket** = pd.cut into [0-12, 13-24, 25-48, 49-72] months. Churn risk is non-linear with tenure — highest in year one, then plateauing. Bucketing encodes this explicitly.

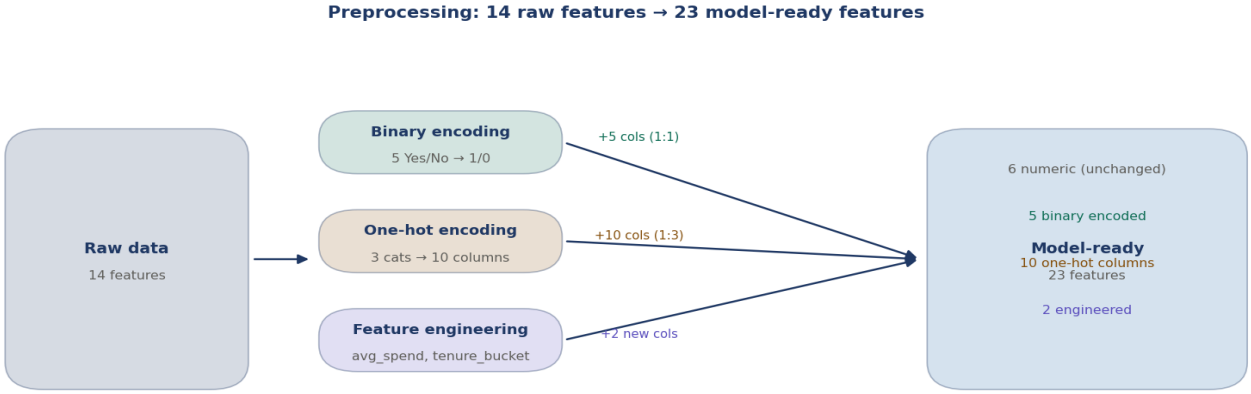

Figure 3 — Preprocessing pipeline: three parallel transformations producing 23 features

Step 3 — Train-Test Split and SMOTE

Stratified split (80/20)

The data is split with stratify=y, ensuring both train and test sets maintain the same 31.6% churn rate. Without stratification, a random split could over- or under-represent churners in one set, giving misleading evaluation metrics:

```
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.2, random_state=42,
stratify=y)
```

SMOTE — Synthetic Minority Over-sampling

The training set has 2,734 retained vs 1,266 churned customers — a 2.16:1 imbalance. If we train on this directly, the model learns to predict 'retained' for almost everyone and still hits ~68% accuracy. SMOTE fixes this by generating synthetic churned customers:

- SMOTE picks a churned customer and finds its K nearest churned neighbours.
- It creates a new synthetic point by interpolating between them in feature space.
- This is repeated until the minority class matches the majority: 2,734 vs 2,734.
- **Critical:** SMOTE is fitted only on X_train. The test set is never touched — applying SMOTE before splitting would leak synthetic patterns into evaluation.

```
sm = SMOTE(random_state=42) X_res, y_res = sm.fit_resample(X_train_scaled, y_train)
```

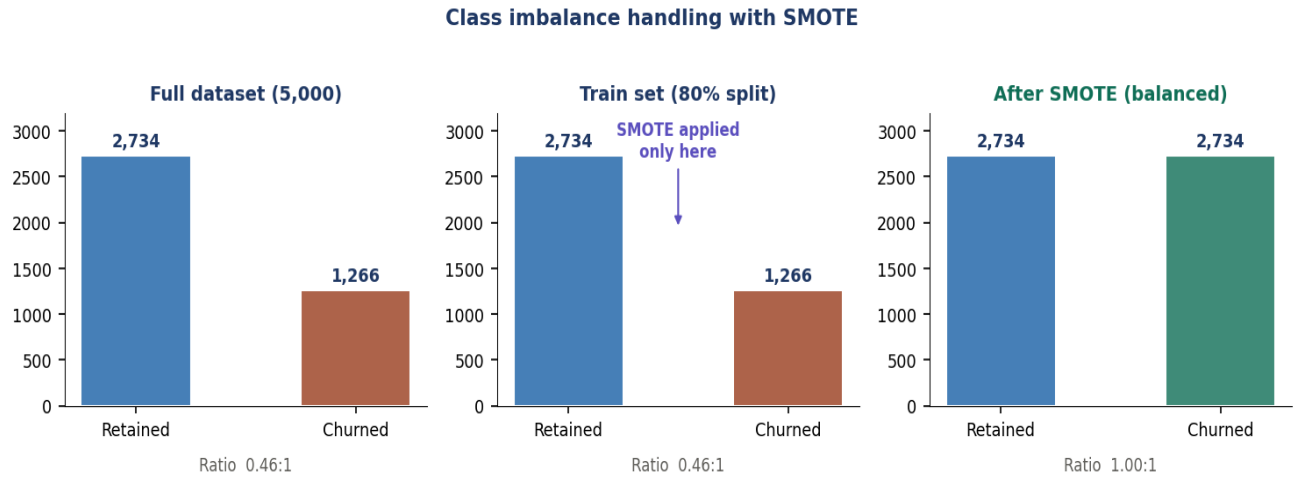

Figure 4 — Class distribution before and after SMOTE (applied only to training data)

Step 4 — Model Training

Two ensemble models are trained on the SMOTE-balanced data. Both are first scaled with StandardScaler — features like total_charges (0–10,000) would otherwise dominate features like senior_citizen (0 or 1).

Random Forest

Builds 300 independent decision trees on random subsets of the data and features, then averages their votes. The randomness prevents individual trees from overfitting. class_weight='balanced' further adjusts for any residual imbalance:

```
RandomForestClassifier(n_estimators=300, max_depth=12, min_samples_leaf=5,
class_weight='balanced')
```

XGBoost

Builds trees sequentially — each tree corrects the errors of all previous trees using gradient descent on the loss function. Subsampling (0.8) and column sampling (0.8) add regularisation to prevent overfitting:

```
XGBClassifier(n_estimators=300, max_depth=6, learning_rate=0.05, subsample=0.8,
colsample_bytree=0.8)
```

Random Forest

Configuration

• Estimators	**300**
• Max depth	**12**
• Class weight	**balanced**
• Scaling	**StandardScaler**

Results

• ROC-AUC	**0.9558**
• F1 Score	**0.8348**
• CV AUC (5-fold)	**0.974 ± 0.003**

XGBoost

Configuration

• Estimators	**300**
• Learning rate	**0.05**
• Subsample	**0.80**
• Col sample	**0.80**

Results

• ROC-AUC	**0.9601**
• F1 Score	**0.8210**
• CV AUC (5-fold)	**0.982 ± 0.002**

Figure 5 — Model configurations and results side by side

Step 5 — Evaluation and Inference

Models are evaluated on the held-out 20% test set (1,000 customers). Four complementary metrics give a full picture of performance:

- **ROC-AUC** — measures how well the model separates churners from non-churners across all probability thresholds. 0.96 is excellent (random = 0.5, perfect = 1.0).
- **Average Precision** — the area under the Precision-Recall curve, focused on the positive (churn) class. More informative than AUC when classes are imbalanced.
- **F1 Score** — harmonic mean of Precision and Recall. RF achieves 0.835, meaning 83.5% of its churn predictions are correct and it catches 88% of actual churners.
- **5-Fold Cross-Validation** — the model is re-trained 5 times on different folds to confirm results aren't a lucky split. XGBoost: 0.982 ± 0.002.

Metric	Random Forest	XGBoost	Notes
ROC-AUC	0.9558	0.9601	XGBoost edges ahead
Average Precision	0.9105	0.9153	Both excellent
F1 Score	0.8348	0.8210	RF slightly better
CV AUC (5-fold)	0.974 ± 0.003	0.982 ± 0.002	XGBoost more stable
Accuracy	89%	88%	Both models strong

Inference demo

A high-risk customer profile (tenure=3 months, monthly charges=■95, month-to-month contract, fiber optic, electronic check, senior citizen, 4 support calls) was passed through both models:

```
Random Forest → churn probability: 99.25% CHURN RISK XGBoost → churn probability: 99.99% CHURN RISK
```

Both models correctly identify this customer as extreme churn risk — the profile triggers every major risk factor simultaneously.

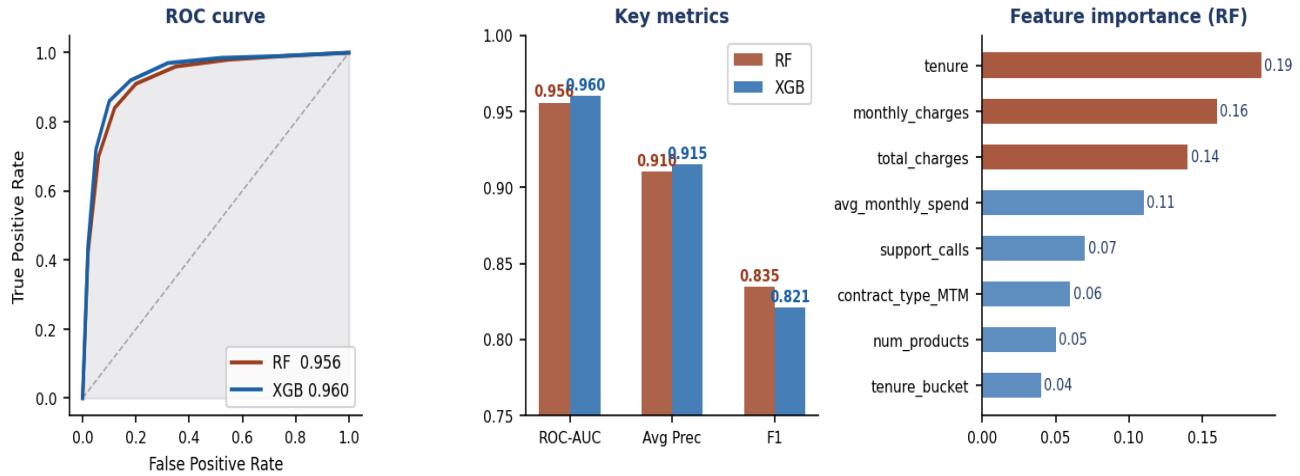

Figure 6 — ROC curves, metric comparison, and top feature importances